KARPAGAM
COLLEGE OF ENGINEERING
Rediscover | Refine | Redefine
Accredited by NAAC with 'A+' grade
Autonomous | Affiliated to Anna University
(An ISO 9001:2015 and ISO 14001:2015 Certified Institution)

ACADEMIC YEAR: 2026-27

ASSIGNMENT FRONT SHEET

Name of the Student:	**AKILESHKUMAR V S**
Register Number:	**717825P302**
Course Code & Title:	**23CSR306 JAVA PROGRAMMING**
Year / Dept. / Sem / Section:	**II / CSE/ III /C**
Assignment:	**II – OOP Mini Project (any one of 35)**
Specification No. & Title:	**Online Auction System**
GitHub Repository Link:	**https://github.com/Akileshkumarvs/online-course-enrollment**
Date of Issue:	
Date of Submission:	

Marks Details

CO / K Level	Maximum Marks	Marks Obtained
CO4 / K3	50	
CO5 / K3	50	
Total	**100**	

Course In-charge

Rubrics for Assignment Evaluation

Assignment Number:	:	II	**Date**	:	
Course Code & Title	:	23CSR306 Java Programming	**Specification No. & Title**	:	Online Auction System
Register Number	:	717825P302	**Name of the Student**	:	AKILESHKUMAR V S
Name of the Evaluator	:		**GitHub Link**	:	https://github.com/Akileshkumarvs/online-course-enrollment

Marks for each criterion = Level (1 – 5) × Weightage. Total: 100 (CO4: 50 + CO5: 50).

Criterion	CO	Wt.	Exemplary (5)	Excellent (4)	Adequate (3)	Average (2)	Inadequate (1)	Max	Marks
Program Correctness & Business Rules	CO4	4	All menu options work; every business rule enforced correctly.	All options work; minor gaps in one rule.	Main flow works; some rules missing.	Partially working; several rules not handled.	Does not run or most features missing.	20	
Input Validation & Exception Handling	CO4	3	All inputs validated; custom exceptions; program never crashes.	Most inputs validated with try-catch.	Basic validation; some invalid inputs not handled.	Little validation; crashes on bad input.	No validation or exception handling.	15	
Menu-driven Interface & Expected Outputs	CO4	3	Clear menu; all expected outputs (bills/receipts/status) shown neatly.	Clear menu; most expected outputs shown.	Menu works; outputs basic or incomplete.	Menu unclear; few outputs shown.	No proper menu or outputs.	15	
OOP Design (Encapsulation, Inheritance, Aggregation, Polymorphism)	CO5	4	All four concepts applied meaningfully; all required classes present.	Concepts applied well; minor design gaps.	Classes present; concepts used in a basic way.	Weak design; some required classes missing.	No OOP design; mostly procedural.	20	
Code Quality & Package Structure	CO5	2	Clean packages, naming, indentation and comments.	Good structure with few lapses.	Acceptable structure; average readability.	Poor naming / structure.	Single file; unreadable code.	10	
GitHub Repository & README	CO5	1	Complete code with clear README (problem, run steps, name, roll no.).	Complete code; README missing minor details.	Code present; README incomplete.	Partial code; no README.	No repository.	5	
Viva / Explanation	CO5	3	Explains design, rules and code clearly; answers all questions.	Explains well; answers most questions.	Explains main logic; answers some questions.	Limited explanation of own code.	Unable to explain the code.	15	
							CO4 Total (Criteria 1 – 3)	**50**	
							CO5 Total (Criteria 4 – 7)	**50**	
							Total Marks	**100**	

Signature of the Evaluator with date

1. Assignment Question

Online Auction System

Problem Statement: Design and implement a Java console application for an Online Auction System that handles sellers, auction items, auctions, bidders, bids, winners, and payment settlement, using MySQL (through JDBC) for storage.

Class Requirements: Seller, AuctionItem, Auction, Bidder, Bid, Winner, Settlement, DBConnection

Business Rules:

1. A seller must be registered before listing an item, and an auction can be created only for an existing item.

2. The end time of an auction must be later than its start time.

3. A bid is accepted only from a registered bidder while the auction is OPEN and the current time is between its start and end time.

4. Every bid must be higher than the current highest bid (or the item's base price when there are no bids yet).

5. An auction can be closed only after its end time; the highest bid (earliest bid on a tie) becomes the winning bid.

6. Payment can be settled only by the winning bidder, after confirmation, and the amount paid must equal the winning price; the auction then becomes SETTLED and the item is marked as transferred.

Menu Options: Add Seller / Add Item / Create Auction / Register Bidder / Place Bid / Close Auction / Settle Payment / Display Auctions / Exit

Expected Output:

- Show confirmation messages with the generated IDs for sellers, items, auctions and bidders.
- Accept a bid only if it is higher than the current highest bid, and show the reason when a bid is rejected.
- Show the winning bidder and winning amount when an auction is closed.
- Show payment settlement confirmation and update the auction status to SETTLED.

2. Java Source Code

Main.java:

Entry point and menu-driven console interface. It connects to MySQL through JDBC and implements the nine menu operations. Bidding, closing an auction and settling payment run as database transactions (SELECT ... FOR UPDATE, commit and rollback), and invalid input or database errors are handled with try-catch blocks. The database password is shown as a placeholder.

```
import java.math.BigDecimal;
import java.sql.*;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.Scanner;
```

```
public class Main {
    private static final String URL =
            "jdbc:mysql://localhost:3306/online_auction";
    private static final String USER = "root";
    private static final String PASSWORD = "YOUR_MYSQL_PASSWORD";
    private static final Scanner sc = new Scanner(System.in);
    private static final DateTimeFormatter FORMAT =
            DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm");
    public static void main(String[] args) {
        try (Connection con = DriverManager.getConnection(
                URL, USER, PASSWORD)) {
            System.out.println("MySQL connected successfully!");
            while (true) {
                System.out.println("\n===== ONLINE AUCTION SYSTEM =====");
                System.out.println("1. Add Seller");
                System.out.println("2. Add Item");
                System.out.println("3. Create Auction");
                System.out.println("4. Register Bidder");
                System.out.println("5. Place Bid");
                System.out.println("6. Close Auction");
                System.out.println("7. Settle Payment");
                System.out.println("8. Display Auctions");
                System.out.println("9. Exit");
                System.out.print("Enter choice: ");
                int choice;
                try {
                    choice = Integer.parseInt(sc.nextLine().trim());
                } catch (NumberFormatException e) {
                    System.out.println("Enter a valid number.");
                    continue;
                }
                try {
                    switch (choice) {
                        case 1 -> addSeller(con);
                        case 2 -> addItem(con);
                        case 3 -> createAuction(con);
                        case 4 -> registerBidder(con);
                        case 5 -> placeBid(con);
                        case 6 -> closeAuction(con);
                        case 7 -> settlePayment(con);
                        case 8 -> displayAuctions(con);
                        case 9 -> {
                            System.out.println("Thank you!");
                            return;
                        }
                        default -> System.out.println("Invalid choice.");
                    }
                } catch (SQLException e) {
```

```
                System.out.println("Database error: " + e.getMessage());
            } catch (IllegalArgumentException e) {
                System.out.println(e.getMessage());
            }
        }
    } catch (SQLException e) {
        System.out.println("Cannot connect to MySQL: "
                + e.getMessage());
        System.out.println(
                "Check MySQL, schema.sql, username, password and JDBC driver.");
    }
}
private static String read(String message) {
    System.out.print(message);
    return sc.nextLine().trim();
}
private static int readInt(String message) {
    return Integer.parseInt(read(message));
}
private static BigDecimal readMoney(String message) {
    BigDecimal amount = new BigDecimal(read(message));
    if (amount.signum() <= 0) {
        throw new IllegalArgumentException(
                "Amount must be greater than zero.");
    }
    return amount;
}
private static LocalDateTime readDate(String message) {
    return LocalDateTime.parse(read(message), FORMAT);
}
private static int insert(Connection con, String sql,
                    Object... values) throws SQLException {
    try (PreparedStatement ps = con.prepareStatement(
            sql, Statement.RETURN_GENERATED_KEYS)) {
        for (int i = 0; i < values.length; i++) {
            ps.setObject(i + 1, values[i]);
        }
        ps.executeUpdate();
        try (ResultSet rs = ps.getGeneratedKeys()) {
            if (rs.next()) return rs.getInt(1);
        }
    }
    throw new SQLException("Could not generate record ID.");
}
private static boolean exists(Connection con, String sql, int id)
        throws SQLException {
    try (PreparedStatement ps = con.prepareStatement(sql)) {
        ps.setInt(1, id);
```

```
            try (ResultSet rs = ps.executeQuery()) {
                return rs.next();
            }
        }
    }
    private static void addSeller(Connection con) throws SQLException {
        String name = read("Seller name: ");
        String contact = read("Seller contact: ");
        int id = insert(con,
                "INSERT INTO sellers(name, contact) VALUES(?, ?)",
                name, contact);
        System.out.println("Seller added. ID: " + id);
    }
    private static void addItem(Connection con) throws SQLException {
        int sellerId = readInt("Seller ID: ");
        if (!exists(con, "SELECT seller_id FROM sellers WHERE seller_id=?",
                sellerId)) {
            System.out.println("Seller not found.");
            return;
        }
        String name = read("Item name: ");
        String description = read("Description: ");
        BigDecimal price = readMoney("Base price: ");
        int id = insert(con,
                "INSERT INTO auction_items"
                + "(seller_id, name, description, base_price) "
                + "VALUES(?, ?, ?, ?)",
                sellerId, name, description, price);
        System.out.println("Item added. ID: " + id);
    }
    private static void createAuction(Connection con) throws SQLException {
        int itemId = readInt("Item ID: ");
        int sellerId;
        try (PreparedStatement ps = con.prepareStatement(
                "SELECT seller_id FROM auction_items WHERE item_id=?")) {
            ps.setInt(1, itemId);
            try (ResultSet rs = ps.executeQuery()) {
                if (!rs.next()) {
                    System.out.println("Item not found.");
                    return;
                }
                sellerId = rs.getInt("seller_id");
            }
        }
        LocalDateTime start = readDate(
                "Start (yyyy-MM-dd HH:mm): ");
        LocalDateTime end = readDate(
                "End   (yyyy-MM-dd HH:mm): ");
```

```
        if (!end.isAfter(start)) {
            System.out.println("End time must be after start time.");
            return;
        }
        int id = insert(con,
                "INSERT INTO auctions"
                + "(item_id, seller_id, start_time, end_time, status) "
                + "VALUES(?, ?, ?, ?, 'OPEN')",
                itemId, sellerId, Timestamp.valueOf(start),
                Timestamp.valueOf(end));
        System.out.println("Auction created. ID: " + id);
    }
    private static void registerBidder(Connection con)
            throws SQLException {
        String name = read("Bidder name: ");
        String contact = read("Bidder contact: ");
        int id = insert(con,
                "INSERT INTO bidders(name, contact) VALUES(?, ?)",
                name, contact);
        System.out.println("Bidder registered. ID: " + id);
    }
    private static void placeBid(Connection con) throws SQLException {
        int auctionId = readInt("Auction ID: ");
        int bidderId = readInt("Bidder ID: ");
        BigDecimal amount = readMoney("Bid amount: ");
        LocalDateTime now = LocalDateTime.now();
        con.setAutoCommit(false);
        try {
            BigDecimal basePrice;
            Timestamp startTime;
            Timestamp endTime;
            String status;
            try (PreparedStatement ps = con.prepareStatement(
                    "SELECT a.start_time, a.end_time, a.status, "
                    + "i.base_price FROM auctions a "
                    + "JOIN auction_items i ON a.item_id=i.item_id "
                    + "WHERE a.auction_id=? FOR UPDATE")) {
                ps.setInt(1, auctionId);
                try (ResultSet rs = ps.executeQuery()) {
                    if (!rs.next()) {
                        throw new SQLException("Auction not found.");
                    }
                    startTime = rs.getTimestamp("start_time");
                    endTime = rs.getTimestamp("end_time");
                    status = rs.getString("status");
                    basePrice = rs.getBigDecimal("base_price");
                }
            }
```

```
        if (!exists(con,
                "SELECT bidder_id FROM bidders WHERE bidder_id=?",
                bidderId)) {
            throw new SQLException("Bidder not found.");
        }
        if (!"OPEN".equals(status)
                || now.isBefore(startTime.toLocalDateTime())
                || !now.isBefore(endTime.toLocalDateTime())) {
            throw new SQLException(
                    "Auction is not currently accepting bids.");
        }
        BigDecimal highest = basePrice;
        try (PreparedStatement ps = con.prepareStatement(
                "SELECT MAX(amount) AS highest FROM bids "
                + "WHERE auction_id=?")) {
            ps.setInt(1, auctionId);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next() && rs.getBigDecimal("highest") != null) {
                    highest = rs.getBigDecimal("highest");
                }
            }
        }
        if (amount.compareTo(highest) <= 0) {
            throw new SQLException(
                    "Bid must be greater than " + highest);
        }
        insert(con,
                "INSERT INTO bids(auction_id, bidder_id, amount) "
                + "VALUES(?, ?, ?)",
                auctionId, bidderId, amount);
        con.commit();
        System.out.println("Bid placed successfully!");
    } catch (SQLException | RuntimeException e) {
        con.rollback();
        throw e;
    } finally {
        con.setAutoCommit(true);
    }
}
private static void closeAuction(Connection con) throws SQLException {
    int auctionId = readInt("Auction ID to close: ");
    con.setAutoCommit(false);
    try {
        Timestamp end;
        String status;
        try (PreparedStatement ps = con.prepareStatement(
                "SELECT end_time, status FROM auctions "
                + "WHERE auction_id=? FOR UPDATE")) {
```

```
            ps.setInt(1, auctionId);
            try (ResultSet rs = ps.executeQuery()) {
                if (!rs.next()) {
                    throw new SQLException("Auction not found.");
                }
                end = rs.getTimestamp("end_time");
                status = rs.getString("status");
            }
        }
        if (!"OPEN".equals(status)) {
            throw new SQLException("Auction is already closed.");
        }
        if (LocalDateTime.now().isBefore(end.toLocalDateTime())) {
            throw new SQLException(
                    "Auction cannot close before its end time.");
        }
        try (PreparedStatement ps = con.prepareStatement(
                "UPDATE auctions SET status='CLOSED' "
                + "WHERE auction_id=?")) {
            ps.setInt(1, auctionId);
            ps.executeUpdate();
        }
        try (PreparedStatement ps = con.prepareStatement(
                "SELECT bid_id, bidder_id, amount FROM bids "
                + "WHERE auction_id=? "
                + "ORDER BY amount DESC, bid_time ASC LIMIT 1")) {
            ps.setInt(1, auctionId);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    try (PreparedStatement win = con.prepareStatement(
                            "INSERT INTO winners"
                            + "(auction_id, bid_id, bidder_id, "
                            + "final_price, confirmed) "
                            + "VALUES(?, ?, ?, ?, FALSE) "
                            + "ON DUPLICATE KEY UPDATE "
                            + "bid_id=VALUES(bid_id), "
                            + "bidder_id=VALUES(bidder_id), "
                            + "final_price=VALUES(final_price), "
                            + "confirmed=FALSE")) {
                        win.setInt(1, auctionId);
                        win.setLong(2, rs.getLong("bid_id"));
                        win.setInt(3, rs.getInt("bidder_id"));
                        win.setBigDecimal(4, rs.getBigDecimal("amount"));
                        win.executeUpdate();
                    }
                    System.out.println("Winning bidder ID: "
                            + rs.getInt("bidder_id"));
                    System.out.println("Winning amount: "
```

```
                        + rs.getBigDecimal("amount"));
                    System.out.println(
                            "Winner confirmation is required before payment.");
                } else {
                    System.out.println("Auction closed with no bids.");
                }
            }
        }
        con.commit();
        System.out.println("Auction closed successfully.");
    } catch (SQLException | RuntimeException e) {
        con.rollback();
        throw e;
    } finally {
        con.setAutoCommit(true);
    }
}
private static void settlePayment(Connection con) throws SQLException {
    int auctionId = readInt("Auction ID: ");
    int bidderId = readInt("Winning bidder ID confirming payment: ");
    BigDecimal paid = readMoney("Amount paid: ");
    con.setAutoCommit(false);
    try {
        BigDecimal finalPrice;
        int winnerId;
        boolean confirmed;
        try (PreparedStatement ps = con.prepareStatement(
                "SELECT bidder_id, final_price, confirmed "
                + "FROM winners WHERE auction_id=? FOR UPDATE")) {
            ps.setInt(1, auctionId);
            try (ResultSet rs = ps.executeQuery()) {
                if (!rs.next()) {
                    throw new SQLException(
                            "Winner not found. Close auction first.");
                }
                winnerId = rs.getInt("bidder_id");
                finalPrice = rs.getBigDecimal("final_price");
                confirmed = rs.getBoolean("confirmed");
            }
        }
        if (winnerId != bidderId) {
            throw new SQLException("Bidder is not the winner.");
        }
        if (!confirmed) {
            String answer = read(
                    "Confirm winner and payment? (yes/no): ");
            if (!answer.equalsIgnoreCase("yes")) {
                con.rollback();
```

```
                System.out.println("Settlement cancelled.");
                return;
            }
        }
        if (paid.compareTo(finalPrice) != 0) {
            throw new SQLException(
                    "Payment must equal winning price: " + finalPrice);
        }
        try (PreparedStatement ps = con.prepareStatement(
                "UPDATE winners SET confirmed=TRUE "
                + "WHERE auction_id=?")) {
            ps.setInt(1, auctionId);
            ps.executeUpdate();
        }
        try (PreparedStatement ps = con.prepareStatement(
                "INSERT INTO settlements"
                + "(auction_id, amount_paid, payment_status, "
                + "item_transfer_status) VALUES(?, ?, 'SUCCESS', "
                + "'TRANSFERRED')")) {
            ps.setInt(1, auctionId);
            ps.setBigDecimal(2, paid);
            ps.executeUpdate();
        }
        try (PreparedStatement ps = con.prepareStatement(
                "UPDATE auctions SET status='SETTLED' "
                + "WHERE auction_id=? AND status='CLOSED'")) {
            ps.setInt(1, auctionId);
            if (ps.executeUpdate() != 1) {
                throw new SQLException(
                        "Auction must be closed before settlement.");
            }
        }
        try (PreparedStatement ps = con.prepareStatement(
                "UPDATE auction_items i JOIN auctions a "
                + "ON i.item_id=a.item_id "
                + "SET i.transferred=TRUE WHERE a.auction_id=?")) {
            ps.setInt(1, auctionId);
            ps.executeUpdate();
        }
        con.commit();
        System.out.println(
                "Payment recorded and item marked as transferred.");
    } catch (SQLException | RuntimeException e) {
        con.rollback();
        throw e;
    } finally {
        con.setAutoCommit(true);
    }
```

```
    }
    private static void displayAuctions(Connection con)
            throws SQLException {
        String sql =
                "SELECT a.auction_id, i.name AS item_name, "
                + "s.name AS seller_name, a.start_time, a.end_time, "
                + "a.status, i.base_price, "
                + "COALESCE(MAX(b.amount), i.base_price) AS highest_bid "
                + "FROM auctions a "
                + "JOIN auction_items i ON a.item_id=i.item_id "
                + "JOIN sellers s ON a.seller_id=s.seller_id "
                + "LEFT JOIN bids b ON a.auction_id=b.auction_id "
                + "GROUP BY a.auction_id, i.name, s.name, "
                + "a.start_time, a.end_time, a.status, i.base_price "
                + "ORDER BY a.auction_id";
        try (PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            System.out.println("\n----- AUCTIONS -----");
            while (rs.next()) {
                System.out.println("\nAuction ID: "
                        + rs.getInt("auction_id"));
                System.out.println("Item: "
                        + rs.getString("item_name"));
                System.out.println("Seller: "
                        + rs.getString("seller_name"));
                System.out.println("Start: "
                        + rs.getTimestamp("start_time"));
                System.out.println("End: "
                        + rs.getTimestamp("end_time"));
                System.out.println("Status: "
                        + rs.getString("status"));
                System.out.println("Base price: "
                        + rs.getBigDecimal("base_price"));
                System.out.println("Highest bid: "
                        + rs.getBigDecimal("highest_bid"));
            }
        }
    }
}
```

Seller.java:

Entity class for a seller (ID, name, contact) with private fields and public getters (encapsulation). Main.java stores the equivalent data in the sellers table.

```
public class Seller {
    private int sellerId;
    private String name;
    private String contact;
```

```
    public Seller(int sellerId, String name, String contact) {
        this.sellerId = sellerId;
        this.name = name;
        this.contact = contact;
    }
    public int getSellerId() { return sellerId; }
    public String getName() { return name; }
    public String getContact() { return contact; }
}
```

AuctionItem.java:

Entity class for an item offered for auction (ID, seller ID, name, description, base price). The data is stored in the auction_items table.

```
public class AuctionItem {
    private int itemId;
    private int sellerId;
    private String name;
    private String description;
    private double basePrice;
    public AuctionItem(int itemId, int sellerId,
                       String name, String description,
                       double basePrice) {
        this.itemId = itemId;
        this.sellerId = sellerId;
        this.name = name;
        this.description = description;
        this.basePrice = basePrice;
    }
    public int getItemId() { return itemId; }
    public int getSellerId() { return sellerId; }
    public String getName() { return name; }
    public String getDescription() { return description; }
    public double getBasePrice() { return basePrice; }
}
```

Auction.java:

Entity class for an auction (ID, item ID, seller ID, start time, end time, status). The status moves through OPEN, CLOSED and SETTLED in the auctions table.

```
public class Auction {
    private int auctionId;
    private int itemId;
    private int sellerId;
    private String startTime;
    private String endTime;
    private String status;
    public Auction(int auctionId,
                   int itemId,
```

```
                int sellerId,
                String startTime,
                String endTime,
                String status) {
        this.auctionId = auctionId;
        this.itemId = itemId;
        this.sellerId = sellerId;
        this.startTime = startTime;
        this.endTime = endTime;
        this.status = status;
    }
    public int getAuctionId() { return auctionId; }
    public int getItemId() { return itemId; }
    public int getSellerId() { return sellerId; }
    public String getStartTime() { return startTime; }
    public String getEndTime() { return endTime; }
    public String getStatus() { return status; }
}
```

Bidder.java:

Entity class for a registered bidder (ID, name, contact), stored in the bidders table.

```
public class Bidder {
    private int bidderId;
    private String name;
    private String contact;
    public Bidder(int bidderId, String name, String contact) {
        this.bidderId = bidderId;
        this.name = name;
        this.contact = contact;
    }
    public int getBidderId() { return bidderId; }
    public String getName() { return name; }
    public String getContact() { return contact; }
}
```

Bid.java:

Entity class for a bid (ID, auction ID, bidder ID, amount), stored in the bids table.

```
public class Bid {
    private long bidId;
    private int auctionId;
    private int bidderId;
    private double amount;
    public Bid(long bidId,
               int auctionId,
               int bidderId,
               double amount) {
        this.bidId = bidId;
```

```
        this.auctionId = auctionId;
        this.bidderId = bidderId;
        this.amount = amount;
    }
    public long getBidId() { return bidId; }
    public int getAuctionId() { return auctionId; }
    public int getBidderId() { return bidderId; }
    public double getAmount() { return amount; }
}
```

Winner.java:

Entity class for the winning bid of a closed auction (auction ID, bid ID, bidder ID, final price, confirmation flag), stored in the winners table.

```
public class Winner {
    private int auctionId;
    private long bidId;
    private int bidderId;
    private double finalPrice;
    private boolean confirmed;
    public Winner(int auctionId,
                  long bidId,
                  int bidderId,
                  double finalPrice,
                  boolean confirmed) {
        this.auctionId = auctionId;
        this.bidId = bidId;
        this.bidderId = bidderId;
        this.finalPrice = finalPrice;
        this.confirmed = confirmed;
    }
    public int getAuctionId() { return auctionId; }
    public long getBidId() { return bidId; }
    public int getBidderId() { return bidderId; }
    public double getFinalPrice() { return finalPrice; }
    public boolean isConfirmed() { return confirmed; }
}
```

Settlement.java:

Entity class for the payment settlement of an auction (ID, auction ID, amount paid), stored in the settlements table.

```
public class Settlement {
    private long settlementId;
    private int auctionId;
    private double amountPaid;
    public Settlement(long settlementId,
                      int auctionId,
                      double amountPaid) {
```

```
        this.settlementId = settlementId;
        this.auctionId = auctionId;
        this.amountPaid = amountPaid;
    }
    public long getSettlementId() { return settlementId; }
    public int getAuctionId() { return auctionId; }
    public double getAmountPaid() { return amountPaid; }
}
```

DBConnection.java:

Helper class that loads the MySQL JDBC driver and returns a database connection. The credentials are placeholders; the password must be replaced with the local MySQL password.

```
import java.sql.Connection;
import java.sql.DriverManager;
public class DBConnection {
    private static final String URL =
            "jdbc:mysql://localhost:3306/online_auction";
    private static final String USER = "root";
    private static final String PASSWORD = "YOUR_MYSQL_PASSWORD";
    public static Connection getConnection() {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            return DriverManager.getConnection(URL, USER, PASSWORD);
        } catch (Exception e) {
            e.printStackTrace();
            return null;
        }
    }
}
```

schema.sql:

Database schema used to run the application. It was written from the tables and columns referenced by the SQL statements in Main.java, because the repository does not contain a schema file.

```
CREATE DATABASE IF NOT EXISTS online_auction;
USE online_auction;
CREATE TABLE sellers (
    seller_id INT AUTO_INCREMENT PRIMARY KEY,
    name      VARCHAR(100) NOT NULL,
    contact   VARCHAR(100) NOT NULL
);
CREATE TABLE auction_items (
    item_id     INT AUTO_INCREMENT PRIMARY KEY,
    seller_id   INT NOT NULL,
    name        VARCHAR(100) NOT NULL,
    description VARCHAR(255),
    base_price  DECIMAL(12,2) NOT NULL CHECK (base_price > 0),
```

```
    transferred BOOLEAN NOT NULL DEFAULT FALSE,
    FOREIGN KEY (seller_id) REFERENCES sellers(seller_id)
);
CREATE TABLE auctions (
    auction_id INT AUTO_INCREMENT PRIMARY KEY,
    item_id    INT NOT NULL,
    seller_id  INT NOT NULL,
    start_time DATETIME NOT NULL,
    end_time   DATETIME NOT NULL,
    status     ENUM('OPEN','CLOSED','SETTLED') NOT NULL DEFAULT 'OPEN',
    FOREIGN KEY (item_id)   REFERENCES auction_items(item_id),
    FOREIGN KEY (seller_id) REFERENCES sellers(seller_id)
);
CREATE TABLE bidders (
    bidder_id INT AUTO_INCREMENT PRIMARY KEY,
    name      VARCHAR(100) NOT NULL,
    contact   VARCHAR(100) NOT NULL
);
CREATE TABLE bids (
    bid_id     BIGINT AUTO_INCREMENT PRIMARY KEY,
    auction_id INT NOT NULL,
    bidder_id  INT NOT NULL,
    amount     DECIMAL(12,2) NOT NULL,
    bid_time   TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
    FOREIGN KEY (auction_id) REFERENCES auctions(auction_id),
    FOREIGN KEY (bidder_id)  REFERENCES bidders(bidder_id)
);
CREATE TABLE winners (
    auction_id  INT PRIMARY KEY,
    bid_id      BIGINT NOT NULL,
    bidder_id   INT NOT NULL,
    final_price DECIMAL(12,2) NOT NULL,
    confirmed   BOOLEAN NOT NULL DEFAULT FALSE,
    FOREIGN KEY (auction_id) REFERENCES auctions(auction_id),
    FOREIGN KEY (bid_id)     REFERENCES bids(bid_id),
    FOREIGN KEY (bidder_id)  REFERENCES bidders(bidder_id)
);
CREATE TABLE settlements (
    settlement_id       BIGINT AUTO_INCREMENT PRIMARY KEY,
    auction_id          INT NOT NULL,
    amount_paid         DECIMAL(12,2) NOT NULL,
    payment_status      VARCHAR(20) NOT NULL,
    item_transfer_status VARCHAR(20) NOT NULL,
    FOREIGN KEY (auction_id) REFERENCES auctions(auction_id)
);
```

3. Execution Output Screenshot

The application was executed with Java 21 against a local MariaDB 10.11 server (MySQL-compatible) using the MySQL Connector/J driver from the repository's lib folder and the schema listed in Section 2. The console output of that run is shown below; the menu that is redisplayed after every action has been omitted from Figures 2 to 7.

```
MySQL connected successfully!

===== ONLINE AUCTION SYSTEM =====
1. Add Seller
2. Add Item
3. Create Auction
4. Register Bidder
5. Place Bid
6. Close Auction
7. Settle Payment
8. Display Auctions
9. Exit

Enter choice: abc
Enter a valid number.

Enter choice: 12
Invalid choice.

Enter choice: 1
Seller name: Ravi Kumar
Seller contact: ravi.kumar@example.com
Seller added. ID: 1

Enter choice: 2
Seller ID: 9
Seller not found.

Enter choice: 2
Seller ID: 1
Item name: Antique Wooden Clock
Description: Vintage 1950s wall clock
Base price: 5000
Item added. ID: 1
```

```
Enter choice: 3
Item ID: 1
Start (yyyy-MM-dd HH:mm): 2026-10-25 10:33
End   (yyyy-MM-dd HH:mm): 2026-10-25 10:30
End time must be after start time.

Enter choice: 3
Item ID: 1
Start (yyyy-MM-dd HH:mm): 2026-10-25 10:33
End   (yyyy-MM-dd HH:mm): 2026-10-25 10:37
Auction created. ID: 1

Enter choice: 4
Bidder name: Anitha S
Bidder contact: anitha@example.com
Bidder registered. ID: 1

Enter choice: 4
Bidder name: Karthik R
Bidder contact: karthik@example.com
Bidder registered. ID: 2
```

```
Enter choice: 5
Auction ID: 1
Bidder ID: 1
Bid amount: 5500
Bid placed successfully!

Enter choice: 5
Auction ID: 1
Bidder ID: 2
Bid amount: 5200
Database error: Bid must be greater than 5500.00

Enter choice: 5
Auction ID: 1
Bidder ID: 2
Bid amount: 6000
Bid placed successfully!

Enter choice: 5
Auction ID: 1
Bidder ID: 99
Bid amount: 7000
Database error: Bidder not found.

Enter choice: 5
Auction ID: 1
Bidder ID: 1
Bid amount: -10
Amount must be greater than zero.
```

```
Enter choice: 6
Auction ID to close: 1
Database error: Auction cannot close before its end time.

Enter choice: 8

----- AUCTIONS -----

Auction ID: 1
Item: Antique Wooden Clock
Seller: Ravi Kumar
Start: 2026-10-25 10:33:00.0
End: 2026-10-25 10:37:00.0
Status: OPEN
Base price: 5000.00
Highest bid: 6000.00

Enter choice: 5
Auction ID: 1
Bidder ID: 1
Bid amount: 8000
Database error: Auction is not currently accepting bids.
```

```
Enter choice: 6
Auction ID to close: 1
Winning bidder ID: 2
Winning amount: 6000.00
Winner confirmation is required before payment.
Auction closed successfully.

Enter choice: 6
Auction ID to close: 1
Database error: Auction is already closed.
```

```
Enter choice: 7
Auction ID: 1
Winning bidder ID confirming payment: 1
Amount paid: 6000
Database error: Bidder is not the winner.

Enter choice: 7
Auction ID: 1
Winning bidder ID confirming payment: 2
Amount paid: 5000
Confirm winner and payment? (yes/no): yes
Database error: Payment must equal winning price: 6000.00

Enter choice: 7
Auction ID: 1
Winning bidder ID confirming payment: 2
Amount paid: 6000
Confirm winner and payment? (yes/no): yes
Payment recorded and item marked as transferred.
```

4. GitHub Repository Link

https://github.com/Akileshkumarvs/online-course-enrollment